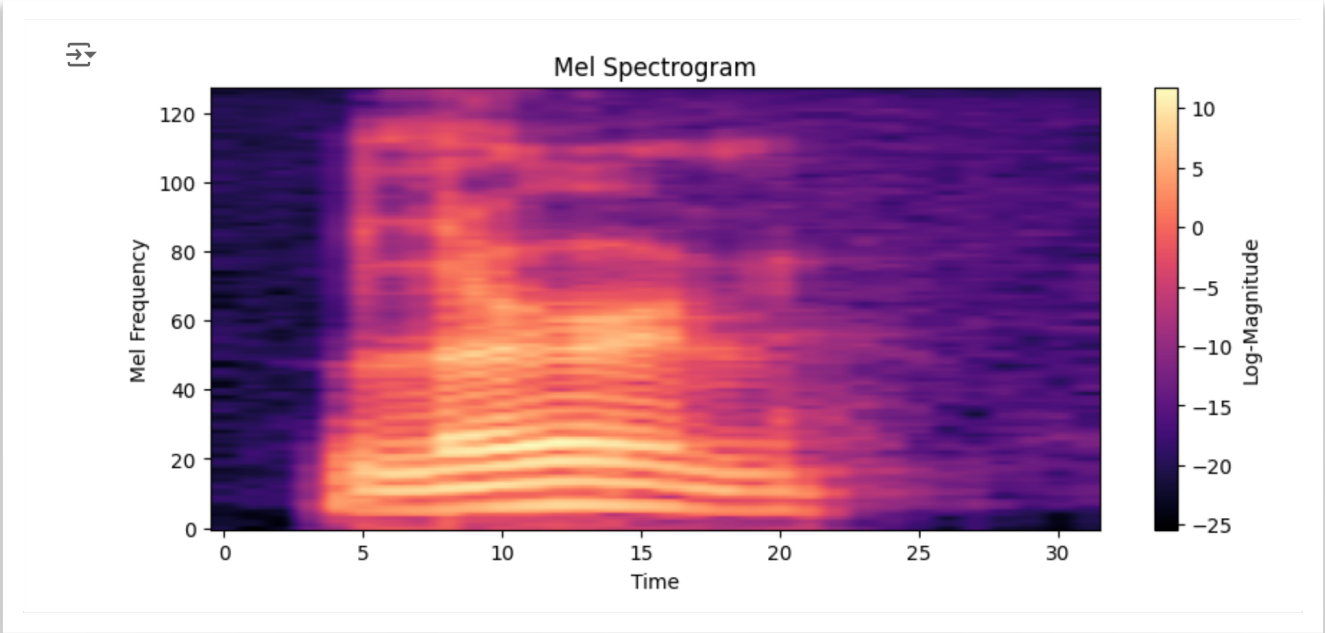
🎯 Project Overview

This project aims to recognize short spoken commands (e.g., “yes”, “no”, “up”, “right”) from raw audio signals.

I built a complete audio classification pipeline, from data preprocessing to model training and evaluation, using the SpeechCommands dataset.

To make audio data usable for deep learning, each waveform was transformed into a **Mel Spectrogram**, a time-frequency representation of sound.
Bright areas correspond to dominant frequencies, while darker regions indicate low-energy signals or silence.



⚙️ Methodology

📊 Dataset

- SpeechCommands dataset
- 35 different voice commands
- Audio clips of ~1 second (16 kHz)

🔄 Preprocessing

- Conversion of raw audio waveforms into **Mel Spectrograms**
- Parameters:
  - Sample rate: 16,000 Hz
  - FFT window: 1024
  - Hop length: 512
  - 128 Mel frequency bands

This transformation allows the model to capture both **temporal and frequency patterns** in audio signals.

🤖 Model Architecture

- 2D Convolutional Neural Network (CNN)
- Multiple convolutional layers + Batch Normalization
- MaxPooling + Adaptive Average Pooling
- Dropout for regularization
- Fully connected output layer (35 classes)

=> The model contains approximately **291,000 parameters** and is specifically designed to process spectrograms as 2D inputs.

📈 Results

- ✅ Best test accuracy: **~61%**
- 📉 Training showed **learning progression but instability**
- ⚠️ Some confusion between similar commands (e.g., "right" vs "up")

=> These results highlight the importance of:

- regularization techniques
- hyperparameter tuning
- and dataset balance

🔍 Key Insights

- Spectrogram-based inputs significantly improve learning compared to raw audio
- 2D CNNs capture both time and frequency patterns effectively
- Model performance is sensitive to:
  - learning rate
  - data distribution
  - and training stability

💡 Applications

This type of model can be used in:

- 🎤 Voice assistants
- 📱 Embedded systems
- 🏭 Industrial voice control systems
- 🤖 Human-machine interaction

🧠 What I Learned

This project allowed me to:

- Build a **complete deep learning pipeline for audio data**
- Understand **signal processing (Mel spectrograms)**
- Design and train a **CNN from scratch**
- Analyze **model performance and limitations**
- Identify issues like **overfitting and class confusion**